# Assignment 4: Prompt Engineering for NLP (NER with Local CoNLL-2003)

## Executive Summary

This assignment explores prompt engineering for **Named Entity Recognition (NER)** using local CoNLL-2003 data files.

I implemented and compared multiple prompt formulations (zero-shot and few-shot) across multiple LLMs, then evaluated entity-level extraction performance and output-format reliability.

Notebook:

- `Assignment4_Prompt Engineering/MingHsiangLee_Assignment4.ipynb`

---

## 1. Dataset and Access Method

### Chosen Dataset

- CoNLL-2003 (local files):

1. `eng.train`
2. `eng.testa`
3. `eng.testb`

### Local Relative Path Used in Notebook

The script uses **relative path candidates only**:

- `Assignment2_Name_Entity_Recognition/conll2003`
- `../Assignment2_Name_Entity_Recognition/conll2003`

This avoids hard-coded absolute paths and keeps the notebook portable in the repository.

---

## 2. What Is 0-shot vs Few-shot?

1. **0-shot prompting**

- The prompt contains only task instructions and the input text.
- No labeled examples are given.

2. **Few-shot prompting**

- The prompt contains instructions plus a small set of labeled examples.
- These examples guide the model to follow output style and labeling behavior.

In this assignment, both styles are tested and compared.

---

## 3. Prompt Designs Tested

I implemented four prompt templates:

1. `zero_shot_plain`
   - Basic instruction with explicit JSON format and a note that text may be ALL CAPS.
   - Adds "No explanation. No code." to prevent hallucination.
2. `zero_shot_rubric`
   - Instruction plus per-type definitions: PER, ORG, LOC, MISC with examples.
   - Clarifies ALL-CAPS input. Adds strict "No code. No prose." rule.
3. `few_shot_caps`
   - Three demonstrations using ALL-CAPS text matching CoNLL's style.
   - Examples: cricket teams (ORG), cities (LOC), country match (LOC).
4. `few_shot_balanced`
   - Three demonstrations including multi-entity, no-entity (`[]`), and mixed cases.
   - ALL-CAPS examples reduce domain mismatch vs. the CoNLL input format.

---

## 4. LLM Requirement: Did We Achieve It?

Yes.

The assignment asks to use an LLM, and gives examples such as GPT, Gemini, or LLaMA. These are examples, not strict requirements.

In this notebook, I used instruction-tuned Google FLAN-T5 LLMs:

| Model | Params | Use case |
|---|---:|---|
| `google/flan-t5-small` | 80M | local CPU baseline |
| `google/flan-t5-base` | 250M | local CPU baseline |
| `google/flan-t5-large` *(optional)* | 780M | Colab T4 GPU — recommended |
| `google/flan-t5-xl` *(optional)* | 3B | Colab A100 GPU — best quality |

How this is achieved technically:

- Models are loaded via Hugging Face `AutoModelForSeq2SeqLM` / `AutoTokenizer`.
- The same NER prompts are sent to each model.
- Outputs are parsed into structured entity predictions.
- Generation uses beam search (`num_beams=4`) for more coherent structured output.

This satisfies the "use LLM and compare model behavior under prompt changes" requirement.

---

## 5. Fast Test vs Full Run Toggle

To support fast debugging and full evaluation, the notebook includes:

- `USE_FAST_MODE = True` for quick tests
- `FAST_SENTENCE_LIMIT = 300` for a meaningful local CPU tuning run
- `FULL_SENTENCE_LIMIT = None` for full-file mode

Behavior:

- If `USE_FAST_MODE` is `True`, only first `FAST_SENTENCE_LIMIT` sentences are evaluated.
- If `USE_FAST_MODE` is `False`, full evaluation is run (or capped by `FULL_SENTENCE_LIMIT` if set).

This provides a clean switch between development and final runs.

---

## 6. Evaluation Protocol

### Output Format

The model is instructed to return only:

- JSON array of objects
- each object has keys `entity` and `type`
- `type` must be one of `PER`, `ORG`, `LOC`, `MISC`

### Metrics

1. **Entity-level Precision**
2. **Entity-level Recall**
3. **Entity-level F1**
4. **Valid JSON Rate**
5. **Exact Sentence Match Rate**

### Comparisons

- Prompt effect (0-shot vs few-shot)
- Model effect (flan-t5-small vs flan-t5-base)

### Latest Local CPU Run Results (Improved)

Run date: 2026-03-17

Configuration used:

- Local CPU
- Evaluated sentences: 300 (`FAST_SENTENCE_LIMIT = 300`)
- Models: `google/flan-t5-small`, `google/flan-t5-base`
- Prompts: `zero_shot_plain`, `zero_shot_rubric`, `few_shot_caps`, `few_shot_balanced`
- Decoding: beam search (`num_beams=4`), `max_new_tokens=128`

Per-prompt metrics:

| Model | Prompt Template | Precision | Recall | F1 | Valid JSON Rate | Exact Sentence Match |
|---|---|---|---|---|---|---|
| google/flan-t5-base | few_shot_caps | 0.8000 | 0.0075 | 0.0150 | 0.0233 | 0.1900 |
| google/flan-t5-small | few_shot_caps | 0.5714 | 0.0075 | 0.0149 | 0.4167 | 0.1900 |
| google/flan-t5-base | few_shot_balanced | 0.5000 | 0.0057 | 0.0112 | 0.0433 | 0.1800 |
| google/flan-t5-small | zero_shot_plain | 0.0000 | 0.0000 | 0.0000 | 0.6633 | 0.1767 |
| google/flan-t5-small | zero_shot_rubric | 0.0000 | 0.0000 | 0.0000 | 0.7967 | 0.1767 |
| google/flan-t5-small | few_shot_balanced | 0.0000 | 0.0000 | 0.0000 | 0.3633 | 0.1767 |
| google/flan-t5-base | zero_shot_plain | 0.0000 | 0.0000 | 0.0000 | 0.3067 | 0.1767 |
| google/flan-t5-base | zero_shot_rubric | 0.0000 | 0.0000 | 0.0000 | 0.1167 | 0.1633 |

0-shot vs few-shot summary from the latest notebook run:

| Model | Shot Type | Precision | Recall | F1 | Valid JSON Rate | Exact Sentence Match |
|---|---|---|---|---|---|---|
| google/flan-t5-base | few-shot | 0.6500 | 0.00660 | 0.01310 | 0.0333 | 0.18500 |
| google/flan-t5-base | zero-shot | 0.0000 | 0.00000 | 0.00000 | 0.2117 | 0.17000 |
| google/flan-t5-small | few-shot | 0.2857 | 0.00375 | 0.00745 | 0.3900 | 0.18335 |
| google/flan-t5-small | zero-shot | 0.0000 | 0.00000 | 0.00000 | 0.7300 | 0.17670 |

Interpretation and evaluation:

- **Pipeline correctness: pass.** Data loading, generation, parsing, aggregation, and logging all run successfully.
- **Entity extraction quality: improved but still low recall.** Non-zero F1 appears for few-shot prompts on both models.
- **Best observed setup:** `google/flan-t5-base` + `few_shot_caps` (F1 = 0.0150, precision = 0.8000, recall = 0.0075).

- **Key tradeoff:** few-shot prompts improve precision but coverage stays very low, so recall remains the bottleneck.
- **Exact sentence match remains low (~0.17-0.19).** Many outputs are still empty (`[]`) or malformed, limiting full-sentence agreement.

Overall judgment for current local CPU setting:

- This is a valid and well-executed prompt-engineering experiment.
- Prompt reformulation produced measurable improvement (from all-zero F1 to non-zero few-shot F1).
- The result is still far from strong NER performance because recall is very low.

---

## 7. Error Analysis of Final Run

Qualitative inspection of logs and parser-source statistics confirms that failures are now mostly coverage-related and format-related.

Parser-source summary (300 samples per model-prompt pair):

- `flan-t5-small zero_shot_rubric`: strict JSON 239 / 300 (79.7%), but mostly empty outputs.
- `flan-t5-small few_shot_caps`: strict JSON 125 / 300 (41.7%) + fallback 7, with non-zero F1.
- `flan-t5-base few_shot_caps`: strict JSON 7 / 300 (2.3%) + fallback 5, but when extracted, precision is high.

Typical remaining failure modes:

| Failure mode | Example output from logs | Effect |
|---|---|---|
| Empty valid JSON | `[]` | Counts as parseable but contributes no true positives |
| Label-only output | `PER`, `LOC` | Not valid entity objects; parser gives empty prediction |
| Near-JSON syntax errors | `["entity": "LONDON", "type": "LOC"]` | Falls to fallback or fails parse |

Key conclusion from error analysis:

- Prompt engineering helped shift behavior from total failure to partial extraction success.
- The dominant limitation is still low entity coverage (recall), not complete parser collapse.

---

## 8. Evidence / Logs

The notebook stores prediction logs for qualitative analysis:

- DataFrame preview in notebook (`logs_df`)
- Timestamped full logs (no overwrite), for example:
  - `assignment4_runs/ner_prompt_output_logs_20260317_134106.csv`
  - `assignment4_runs/metrics_20260317_134106.csv`
  - `assignment4_runs/shot_summary_20260317_134106.csv`

    - `assignment4_runs/parse_source_summary_20260317_134106.csv`
- Latest convenience snapshots:
    - `assignment4_runs/ner_prompt_output_logs_latest.csv`
    - `assignment4_runs/metrics_latest.csv`
    - `assignment4_runs/shot_summary_latest.csv`
    - `assignment4_runs/parse_source_summary_latest.csv`

This satisfies the requirement to show output differences for different prompt formulations.

---

## 9. Bias Risks and Mitigation

### Potential Biases

1. Over-representation of certain name/geography patterns in news corpora.
2. Prompt-example bias toward specific entity types.
3. Domain mismatch when moving from news text to other domains.

### Mitigation

1. Use balanced few-shot examples across entity types.
2. Include no-entity examples (`[]`) to reduce over-tagging.
3. Evaluate slices by entity type and domain.
4. Keep strict output schemas with parser checks.

---

## 10. Conclusion

This assignment demonstrates prompt engineering for NER with a local CoNLL-2003 dataset, relative-path loading, model comparison, and fast/full evaluation toggle.

Current status after latest local CPU run:

- Functional status: pass
- Quality status on current setup: partial success (few-shot prompts produce non-zero F1, but recall is still very low)

Deliverables covered:

1. Jupyter notebook with multiple prompt designs: complete
2. Comparative analysis framework: complete
3. Logs for different outputs across prompt formulations: complete
4. Bias discussion and mitigation: complete
5. Multi-model comparison under same prompt setting: complete

---

## 11. Submission Package

Recommended files to submit:

1. Main notebook:

    - `Assignment4_Prompt Engineering/MingHsiangLee_Assignment4.ipynb`
2. Report:
    - `Assignment4_Prompt Engineering/MingHsiangLee_Assignment4_report.md`
3. Prompt archive (old vs new prompts):
    - `Assignment4_Prompt Engineering/prompt_formulations_old_vs_new.txt`
4. Supporting evidence logs (latest run):
    - `Assignment4_Prompt Engineering/assignment4_runs/metrics_latest.csv`
    - `Assignment4_Prompt Engineering/assignment4_runs/shot_summary_latest.csv`
    - `Assignment4_Prompt Engineering/assignment4_runs/ner_prompt_output_logs_latest.csv`
    - `Assignment4_Prompt Engineering/assignment4_runs/parse_source_summary_latest.csv`